

JavaScript Object Notation and JSON Signatures

Web Security
Juraj Somorovsky

Summer Semester 2024
Paderborn University

Overview

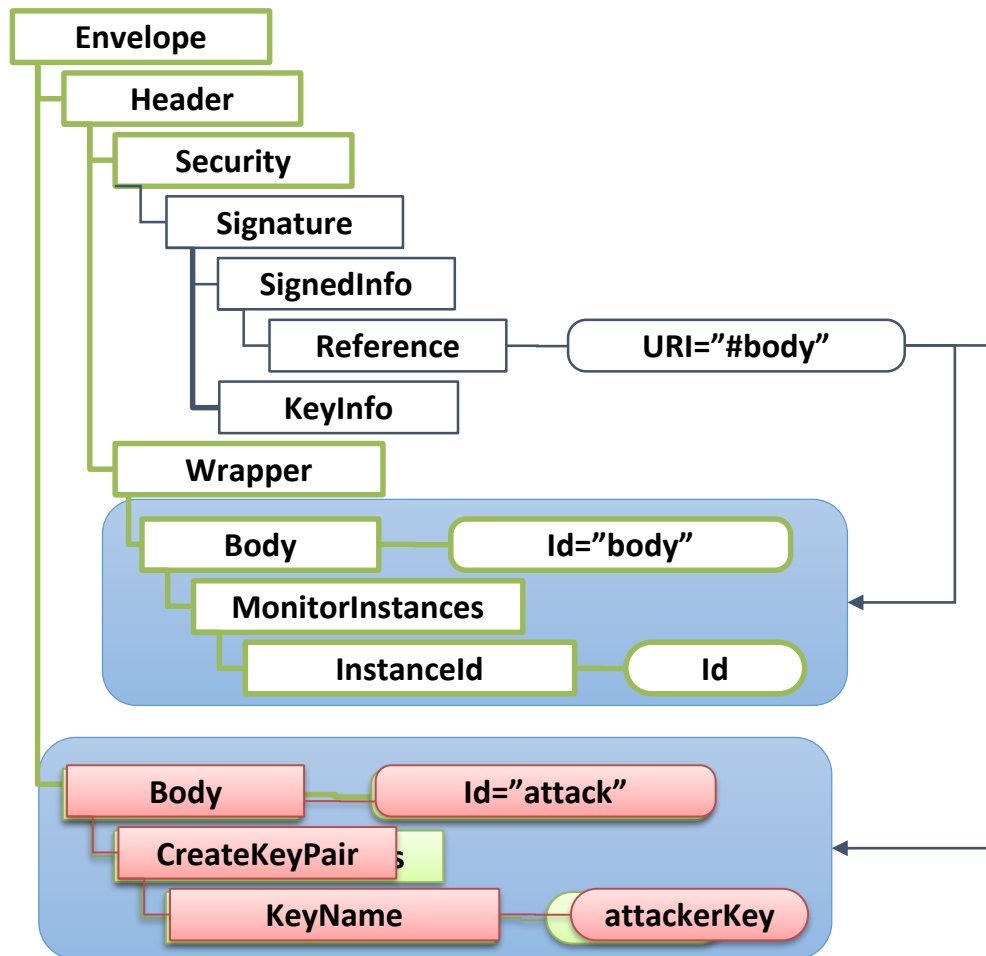
1. Web technologies
2. Web attacks
3. XML and Single Sign-On (SAML)
4. JSON and OIDC / OAuth
5. JSON Web Services

XML Security

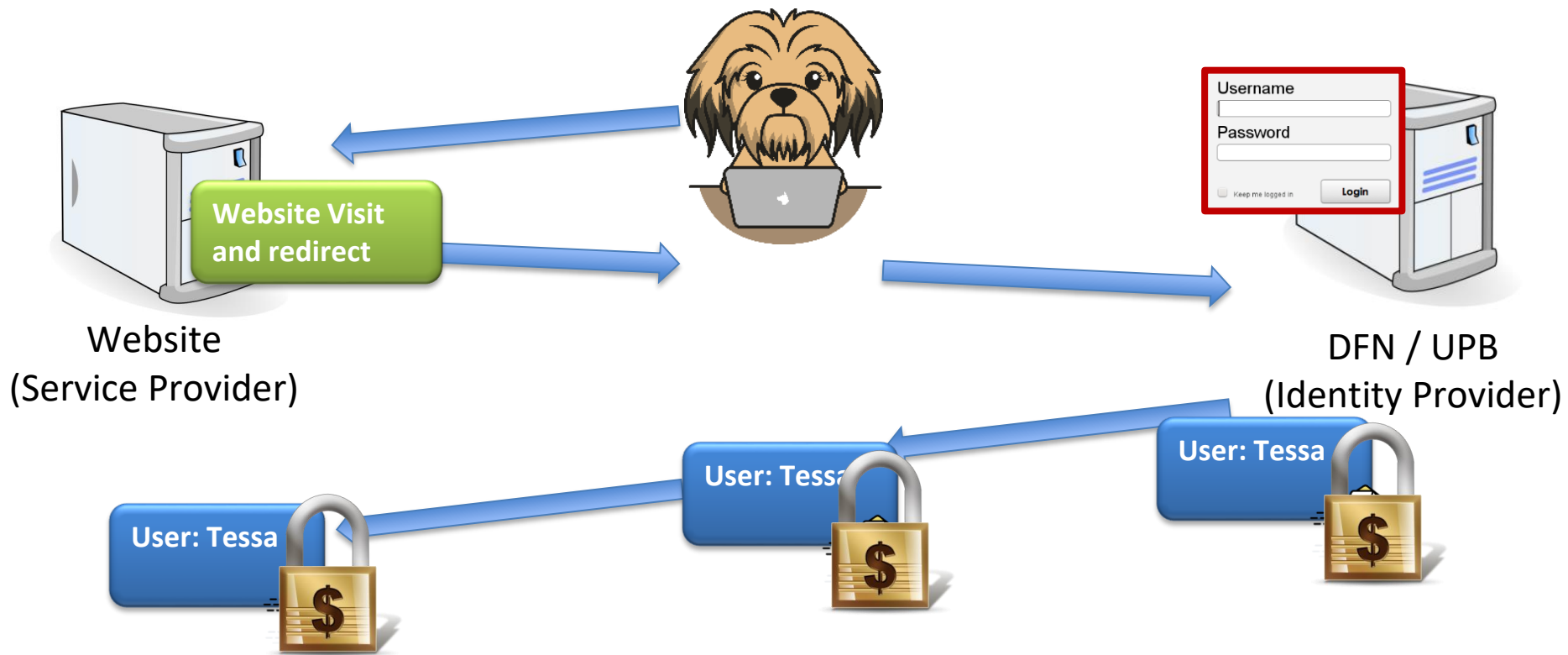
- W3C Standard for securing XML messages
- XML Encryption: protects confidentiality
- XML Signature: protects authenticity and integrity
- Very flexible

```
<PaymentInfo>  
  <Name>John Smith</Name>  
  <CreditCard Limit='5,000'>  
    <Number>4019 ...5567</Number>  
    <Issuer>Example Bank</Issuer>  
    <Expiration>04/02</Expiration>  
  </CreditCard>  
</PaymentInfo>
```

XML Signature Wrapping



SAML-based Single Sign-On



Evaluation Results

Service Provider	AM1				RA	AM2		TRC	AM3 CInj	Summary
	ØSig	CF	XXEA	XSLTA		XSW				
Salesforce	×	×	×	×	×	×	×	×	×	×
Google Apps	×	×	×	×	×	×	×	×	×	×
Zoho	×	×	×	×	×	✓	×	×	×	✓
Zendesk	×	×	×	×	×	×	✓	×	×	✓
Clarizen	✓	×	✓	×	✓	✓	✓	×	×	✓
SAMange	×	×	✓	×	×	✓	✓	✓	✓	✓
Shiftplanning	×	×	✓	×	×	×	✓	✓	✓	✓
Panorama9	×	×	×	×	×	×	✓	×	×	✓
UserVoice (Marketing)	×	×	×	×	×	×	✓	×	×	✓
Instructure	×	×	×	✓	✓	✓	✓	×	×	✓
The Resumator	×	×	✓	×	×	×	✓	×	×	✓
BambooHR	×	×	×	×	×	×	✓	✓	✓	✓
AppDynamics	×	×	✓	×	✓	✓	✓	×	×	✓
IdeaScale	×	×	✓	×	×	×	×	✓	✓	✓
Panopto	×	×	×	×	×	✓	✓	×	×	✓
TimeOffManager	×	×	✓	×	✓	✓	✓	×	×	✓
HappyFox	×	×	×	×	×	✓	✓	×	×	✓
SpringCM	×	×	×	×	×	✓	×	×	×	✓
ScreenSteps Live	×	×	✓	×	×	✓	✓	×	×	✓
LiveHive	×	×	✓	×	✓	✓	✓	×	×	✓
Howlr	×	×	×	×	×	×	✓	✓	✓	✓
CA Service Management	×	×	✓	×	✓	×	✓	✓	✓	✓
Total	1	0	10	1	6	11	17	6		20/ 22

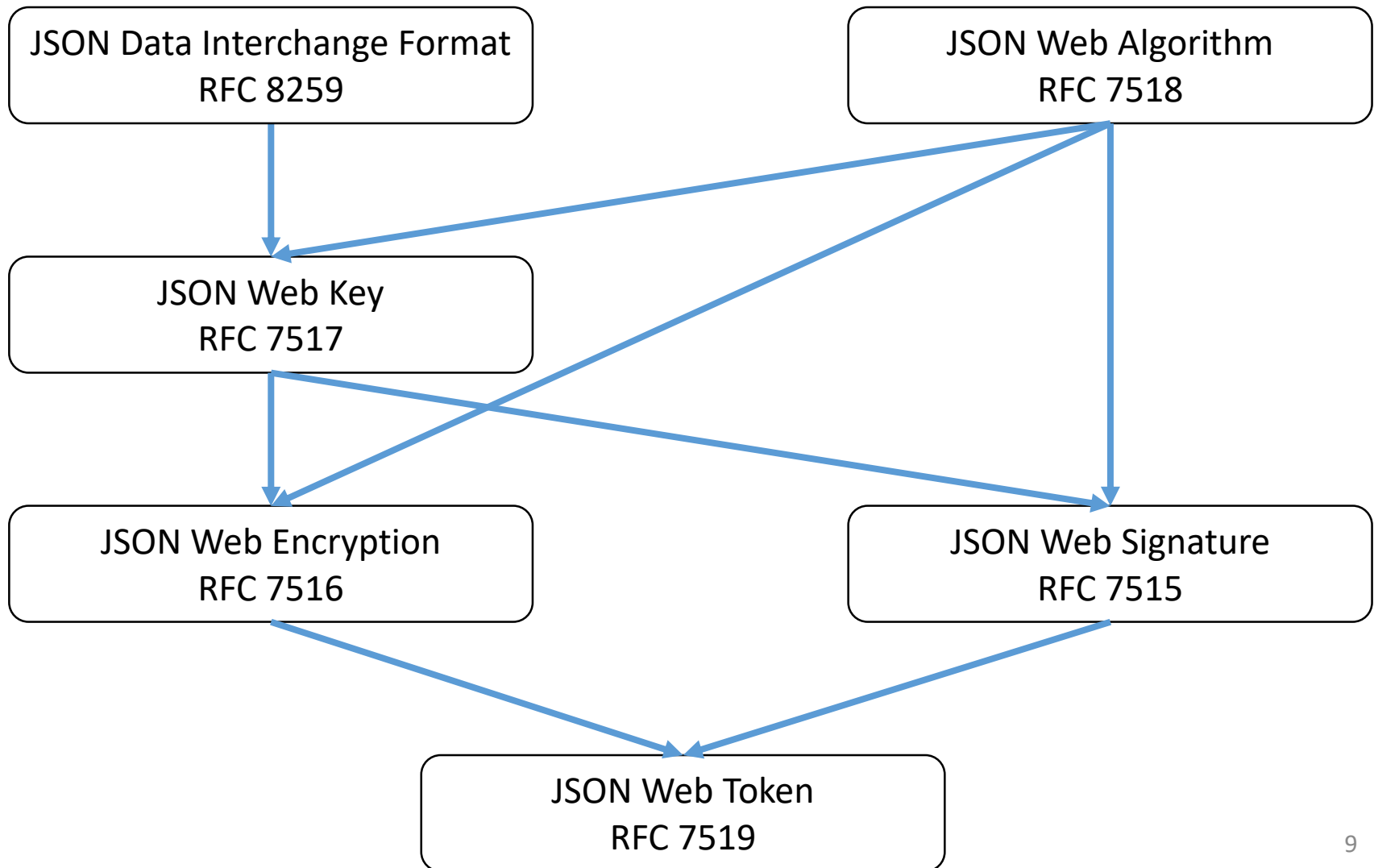
Today

1. Web technologies
2. Web attacks
3. XML and Single Sign-On (SAML)
- 4. JSON and OIDC / OAuth**
5. PDF security

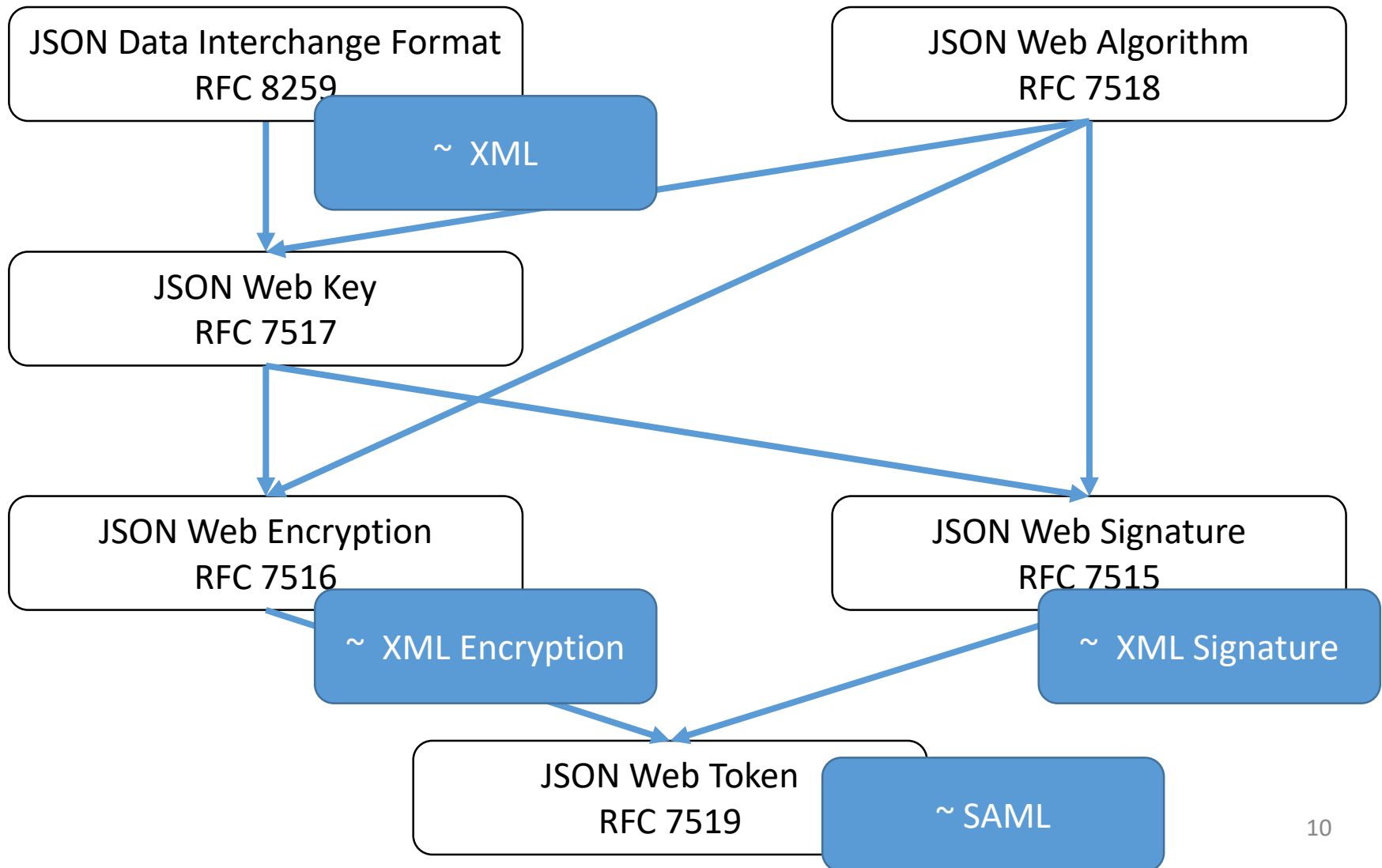
Today

- What is JSON?
- Where and for what is it used?
- Which security features does it offer?
- JSON crypto (signatures)
- What can go wrong?

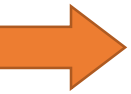
Overview: JavaScript Object Signing and Encryption (JOSE) working group



Overview: JavaScript Object Signing and Encryption (JOSE) working group



Overview

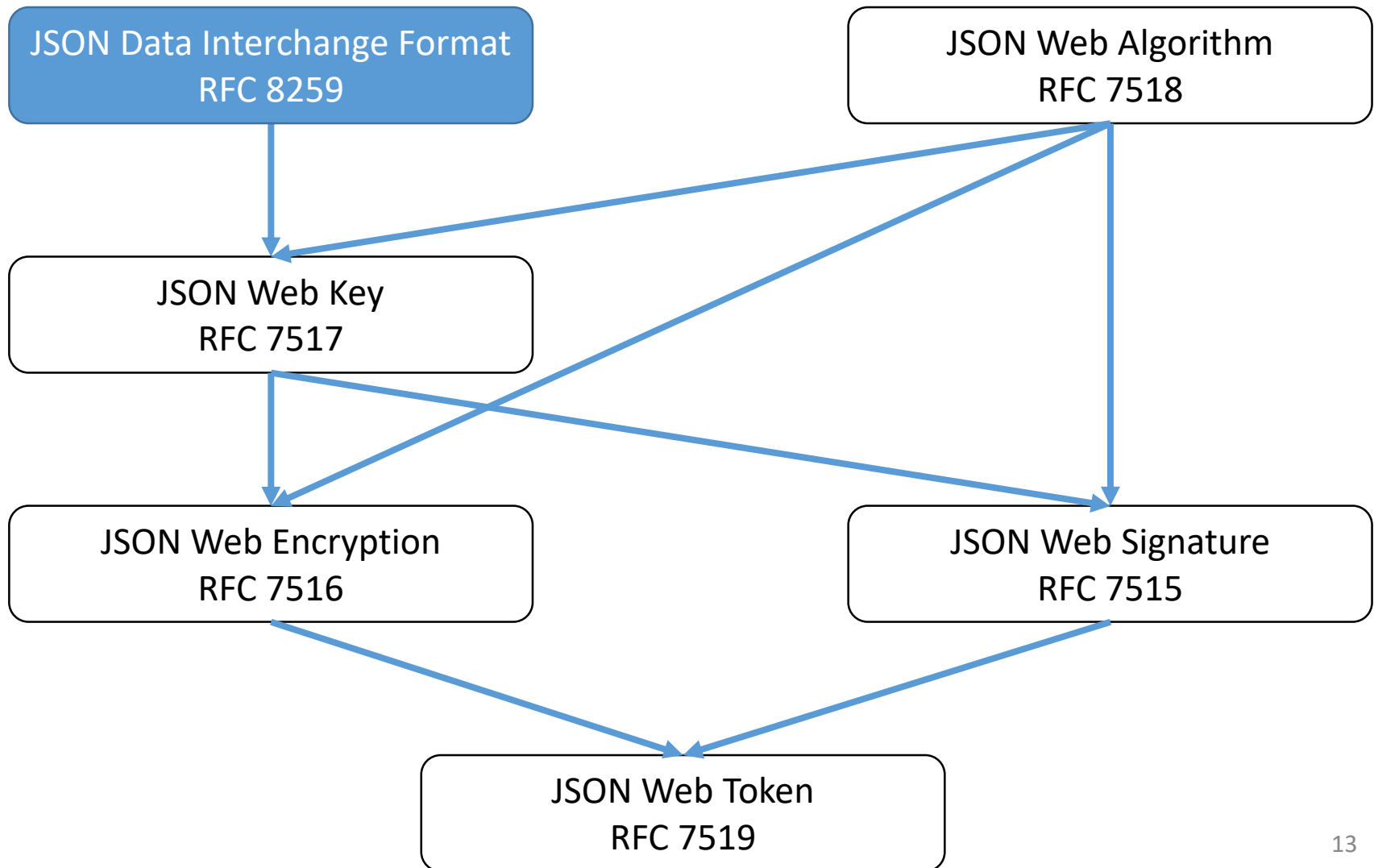


1. JSON
2. JSON Crypto
3. Attacks

Let's start

```
{"JSON": "Overview"}
```

Overview: JavaScript Object Signing and Encryption (JOSE) working group



The JavaScript Object Notation (JSON) Data Interchange Format

- lightweight, text-based, language-independent data interchange format
- Four *primitives*
 - *strings*, *numbers*, *booleans*, *null*
- Two structure types
 - *objects*: {}
 - Unordered collection of *name: value* pairs
 - *name*: *string*
 - *value*: *primitive* or *object* or *array*
 - *arrays*: []
 - Ordered sequence of zero or more *values*

Show me some JSON Examples!

- ```
{ "name": "Web Security",
 "cp": 6,
 "tags": ["web", "xml", "json", "sso", "pdf"]
}
```
- ```
["Vladi", "Christian", "Juraj"]
```
- ```
[
 { "bachelor": false, "master": true },
 { "misc": null }
]
```

# Parsing JSON in Browser (Firefox)

```
>> lecture=JSON.parse('{ "name": "Web Security", "cp": 5, "tags": ["web", "xml", "json", "sso", "pdf"] }')
```

```
← ▼ Object { name: "Web Security", cp: 5, tags: (5) [...] }
```

```
 cp: 5
```

```
 name: "Web Security"
```

```
 ▼ tags: Array(5) ["web", "xml", "json", ...]
```

```
 0: "web"
```

```
 1: "xml"
```

```
 2: "json"
```

```
 3: "sso"
```

```
 4: "pdf"
```

```
 length: 5
```

```
 ▶ <prototype>: Array []
```

```
 ▶ <prototype>: Object { ... }
```

```
>> lecture.name
```

```
← "Web Security"
```

```
>> lecture.tags[2]
```

```
← "json"
```



# Valid or Invalid?



Let's play a (parsing)  
game...

# Valid or Invalid?

- `JSON.parse('{}')`
  - `{}` // valid
  - `typeof(JSON.parse('{}'))`  
// "object"
- Same for:
  - `JSON.parse('[]')` // "object"

# Valid or Invalid?

- `JSON.parse('null')`
  - `null // valid`
    - Different to `{}` !
  - `typeof(JSON.parse('null'))`  
`// "object"`

# Valid or Invalid?

- `JSON.parse('True')`

```
✖ ▶ Uncaught SyntaxError: Unexpected token T in JSON at position 0
 at JSON.parse (<anonymous>)
 at <anonymous>:1:6
```

- But we can parse each primitive:
  - `JSON.parse('true') // "boolean"`
  - `JSON.parse('false') // "boolean"`
  - `JSON.parse('42') // "number"`
  - `JSON.parse('"some test string"')  
 // "string"`

# Valid or Invalid?

- `JSON.parse(' "lecture": "Web" ')`

✖ ▶ Uncaught SyntaxError: Unexpected token : in JSON at position 9  
at JSON.parse (<anonymous>)  
at <anonymous>:1:10

# Valid or Invalid?

- `JSON.parse('{lecture: "Web"}')`

✖ ▶ Uncaught SyntaxError: Unexpected token l in JSON at position 1  
at JSON.parse (<anonymous>)  
at <anonymous>:1:10

- Note: this would be a valid "object literal"
  - Maybe you will see such definitions in JavaScript
  - But it is not a valid JSON

# Valid or Invalid?

- `JSON.parse('{42}')`

✖ ▶ Uncaught SyntaxError: Unexpected number in JSON at position 1  
at JSON.parse (<anonymous>)  
at <anonymous>:1:10

# Valid or Invalid?

- `JSON.parse('[42]')`
  - `[42] // valid`
- `typeof(JSON.parse('[42]'))`  
`// "object"`
  - `typeof(JSON.parse([])) // "object"`



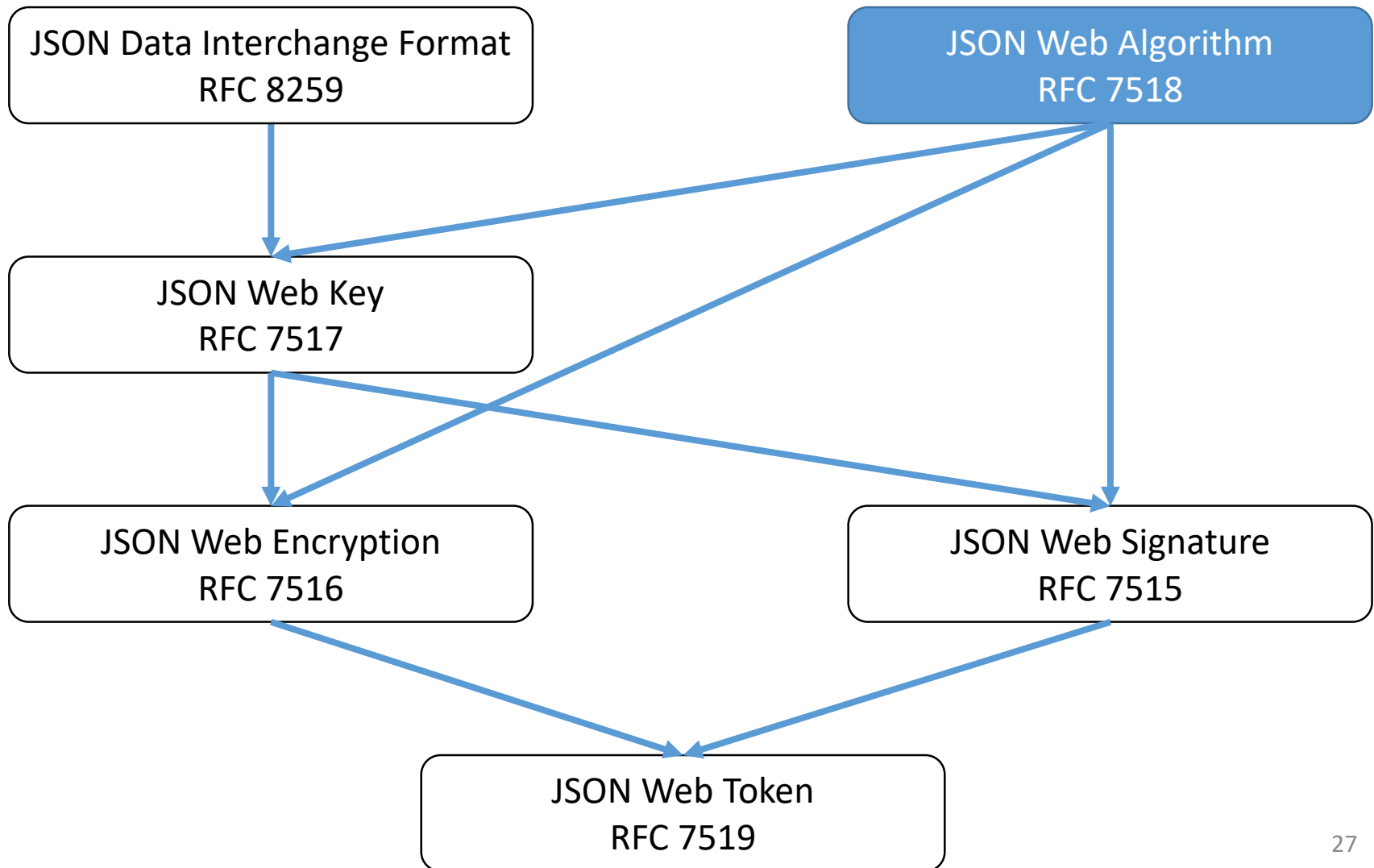
# Questions



# Overview

1. JSON
-  2. JSON Crypto
3. Attacks

# Overview: JavaScript Object Signing and Encryption (JOSE) working group



# JSON Web Algorithm (RFC 7518)



- Registry of cryptographic algorithms...

1. ... for Key Management
2. ... for Digital Signatures and MACs
3. ... for Content Encryption
4. ... for Keys (Parameters for Keys)

**Let's take a look at some examples...**

# 1. Crypto Algorithms for Key Management

- "alg" (Algorithm) parameter Values for JWE

| "alg" Param Value | Key Management Algorithm                                  | More Header Params | Implementation Requirements |
|-------------------|-----------------------------------------------------------|--------------------|-----------------------------|
| RSA1_5            | RSAES-PKCS1-v1_5                                          | (none)             | Recommended-                |
| RSA-OAEP          | RSAES OAEP using default parameters                       | (none)             | Recommended+                |
| RSA-OAEP-256      | RSAES OAEP using SHA-256 and MGF1 with SHA-256            | (none)             | Optional                    |
| A128KW            | AES Key Wrap with default initial value using 128-bit key | (none)             | Recommended                 |

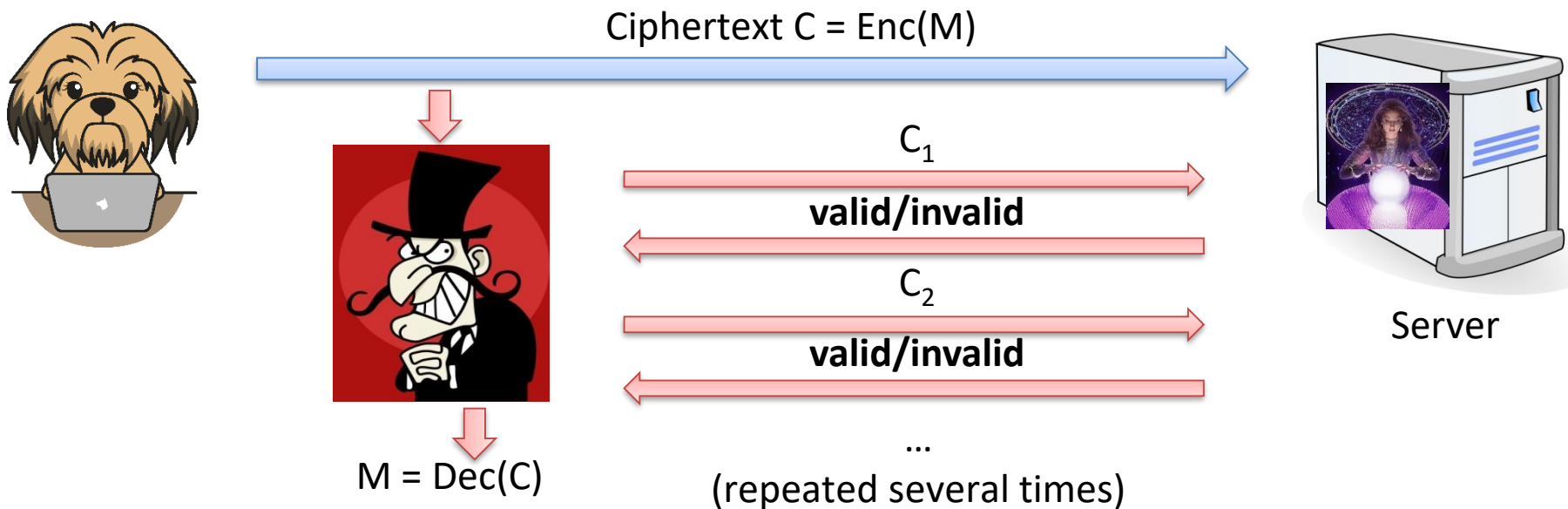


Problems?

**Bleichenbacher's attack!**  
(not going to handle in this lecture)

# Bleichenbacher's Attack

- 1998: Adaptive chosen-ciphertext attack
- Exploits strict **RSA PKCS#1 v1.5** padding validation
  - And a side channel provided by the server



## 2. Crypto Algorithms for Signatures & MACs

- „alg“ (Algorithm) parameter for JWS

| "alg" Param Value | Digital Signature or MAC Algorithm             | Implementation Requirements |
|-------------------|------------------------------------------------|-----------------------------|
| HS256             | HMAC using SHA-256                             | Required                    |
| HS384             | HMAC using SHA-384                             | Optional                    |
| HS512             | HMAC using SHA-512                             | Optional                    |
| RS256             | RSASSA-PKCS1-v1_5 using SHA-256                | Recommended                 |
| RS384             | RSASSA-PKCS1-v1_5 using SHA-384                | Optional                    |
| RS512             | RSASSA-PKCS1-v1_5 using SHA-512                | Optional                    |
| ES256             | ECDSA using P-256 and SHA-256                  | Recommended+                |
| ES384             | ECDSA using P-384 and SHA-384                  | Optional                    |
| ES512             | ECDSA using P-521 and SHA-512                  | Optional                    |
| PS256             | RSASSA-PSS using SHA-256 and MGF1 with SHA-256 | Optional                    |
| PS384             | RSASSA-PSS using SHA-384 and MGF1 with SHA-384 | Optional                    |
| PS512             | RSASSA-PSS using SHA-512 and MGF1 with SHA-512 | Optional                    |
| none              | No digital signature or MAC performed          | Optional                    |



Problems?

# Using the Algorithm "none"

JWSs MAY also be created that do not provide integrity protection. Such a JWS is called an Unsecured JWS. An Unsecured JWS uses the "alg" value "none" and is formatted identically to other JWSs, but MUST use the empty octet sequence as its JWS Signature value. Recipients MUST verify that the JWS Signature value is the empty octet sequence.

Implementations that support Unsecured JWSs MUST NOT accept such objects as valid unless the application specifies that it is acceptable for a specific object to not be integrity protected. Implementations MUST NOT accept Unsecured JWSs by default. In order to mitigate downgrade attacks, applications MUST NOT signal acceptance of Unsecured JWSs at a global level, and SHOULD signal acceptance on a per-object basis. See [Section 8.5](#) for security considerations associated with using this algorithm.

<https://tools.ietf.org/html/rfc7518#section-3.6>





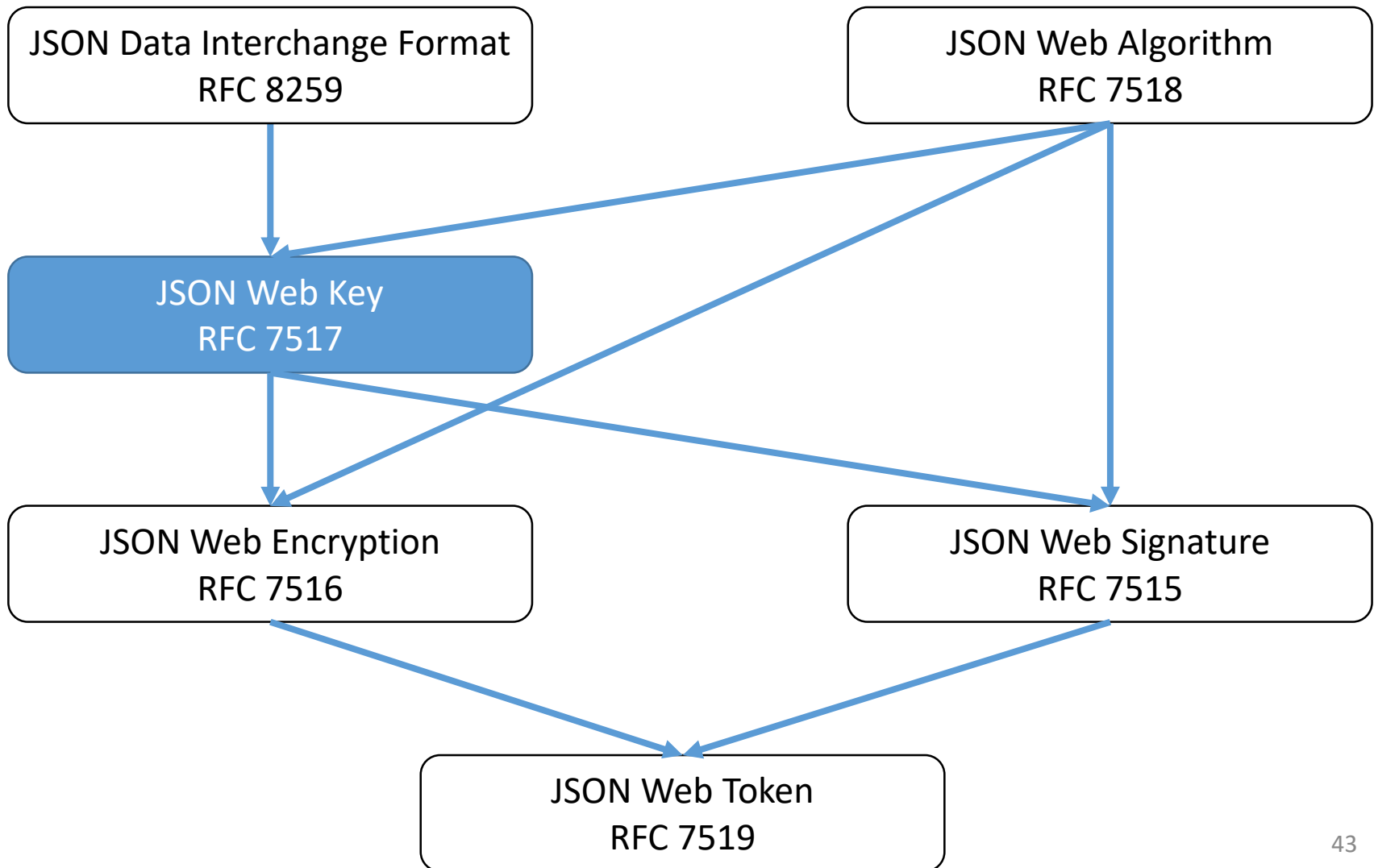
# Using the Algorithm "none"

JWSs MAY also be created that do not provide integrity protection. Such a JWS is **called an Unsecured JWS**. An Unsecured JWS uses the **"alg" value "none"** and is formatted identically to other JWSs, but MUST use the **empty octet sequence as its JWS Signature value**. Recipients MUST verify that the JWS Signature value is the empty octet sequence.

Implementations that support Unsecured JWSs MUST NOT accept such objects as valid unless the application specifies that it is acceptable for a specific object to not be integrity protected. **Implementations MUST NOT accept Unsecured JWSs by default**. In order to mitigate downgrade attacks, applications MUST NOT signal acceptance of Unsecured JWSs at a global level, and SHOULD signal acceptance on a per-object basis. See [Section 8.5](#) for security considerations associated with using this algorithm.

<https://tools.ietf.org/html/rfc7518#section-3.6>

# Overview: JavaScript Object Signing and Encryption (JOSE) working group



# JSON Web KeySet Example

```
1. {
2. "keys": [
3. {
4. "kty": "oct",
5. "alg": "A128KW",
6. "k": "Abwt...",
7. },
8. {
9. "kty": "EC",
10. "crv": "P-256",
11. "x": "D4vAi...",
12. "y": "UsDy...",
13. "use": "enc",
14. "kid": "1"
15. },
16. {
17. "kty": "RSA",
18. "n": "kAnx...",
19. "e": "aycD...",
20. "alg": "RS256",
21. "kid": "202"
22. }
23.]
24. }
```

- Symmetric key
- AES128 Key Wrapping
- Key Value
- Elliptic Curve
- NIST Named Curve P-256
- X/Y coordinates on EC
- Can be “enc” or “sig”
- Unique string to identify the key
- RSA Key Type
- RSA Parameters
- RSASSA-PKCS1-v1\_5 using SHA-256

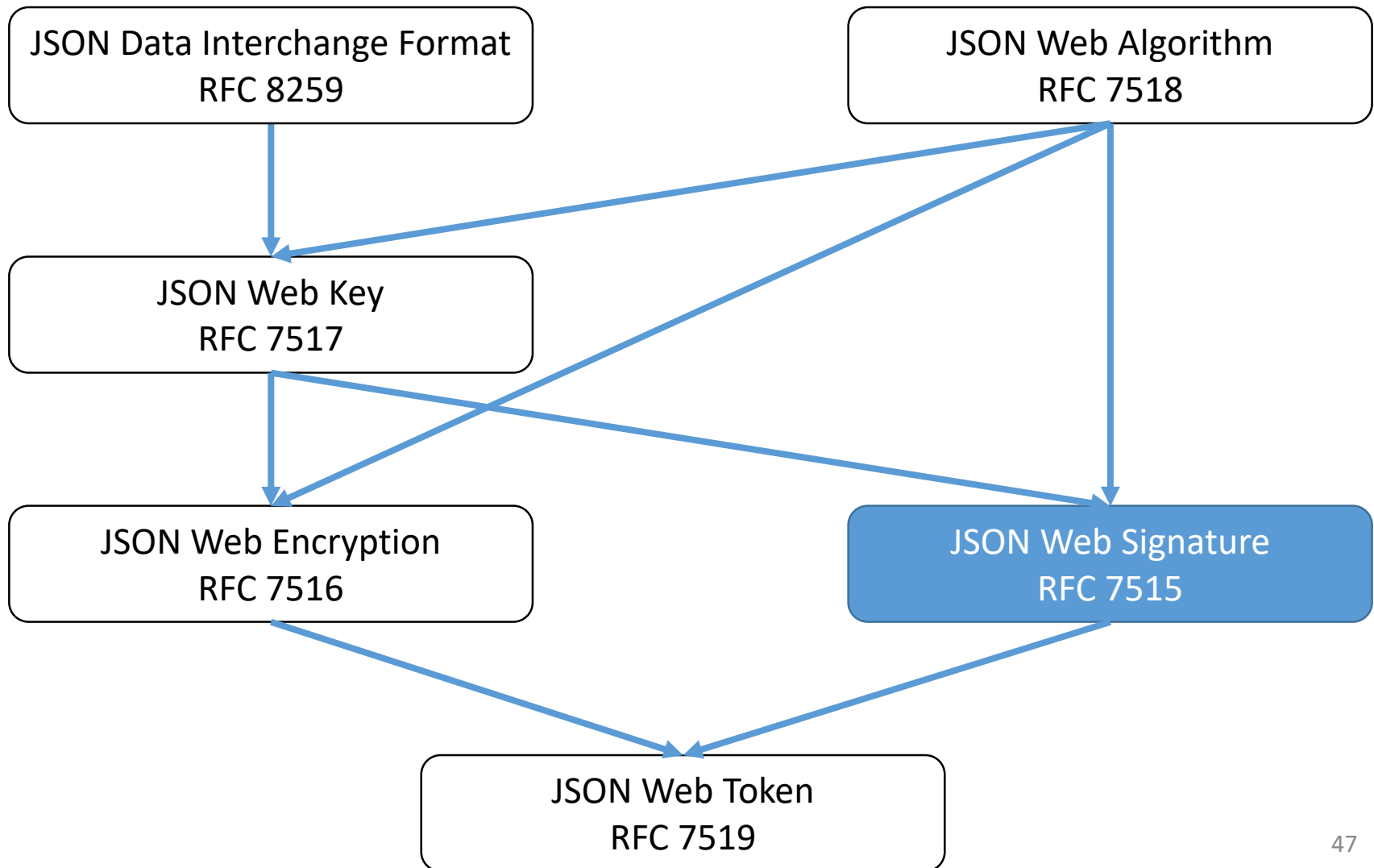
# JSON Web Key

- Single Key: {"kty":"EC",...}
- Key Set: {"keys": [...]}
- "kty" – key type, "alg" – algorithm
  - See JSON Web Algorithm
- "use" – key use: "sig", "enc",
- "key\_ops" – Key Operations: different use cases, e.g. key wrapping, derive key, ...

# JSON Web Key

- "x5u" (X.509 URL) Header Parameter
- "x5c" (X.509 Certificate Chain) Header Parameter
- "x5t" (X.509 Certificate SHA-1 Thumbprint) Header Parameter
- "kid" (Key ID) Header Parameter

# Overview: JavaScript Object Signing and Encryption (JOSE) working group

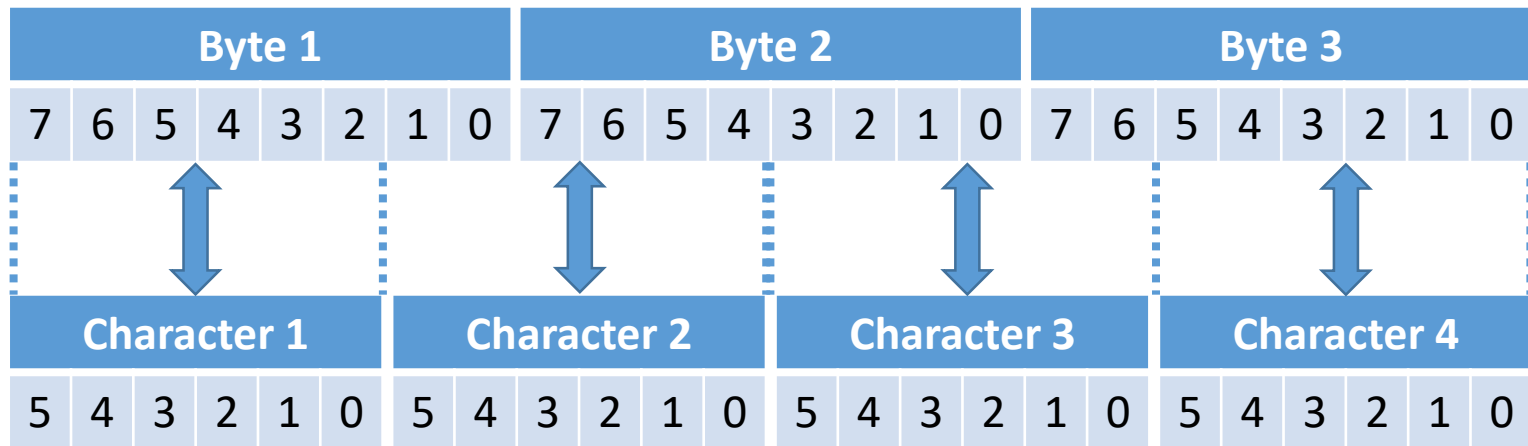


# JSON Web Signature

- Basically a Framework to sign arbitrary JSON payloads
  - Signs the whole payload
    - Similar to XML Signatures...but much simpler
  - Consists (in most cases) of three parts:
    1. Header contains signature information
    2. Payload
    3. Signature Value

# Base64

- Binary to printable text encoding



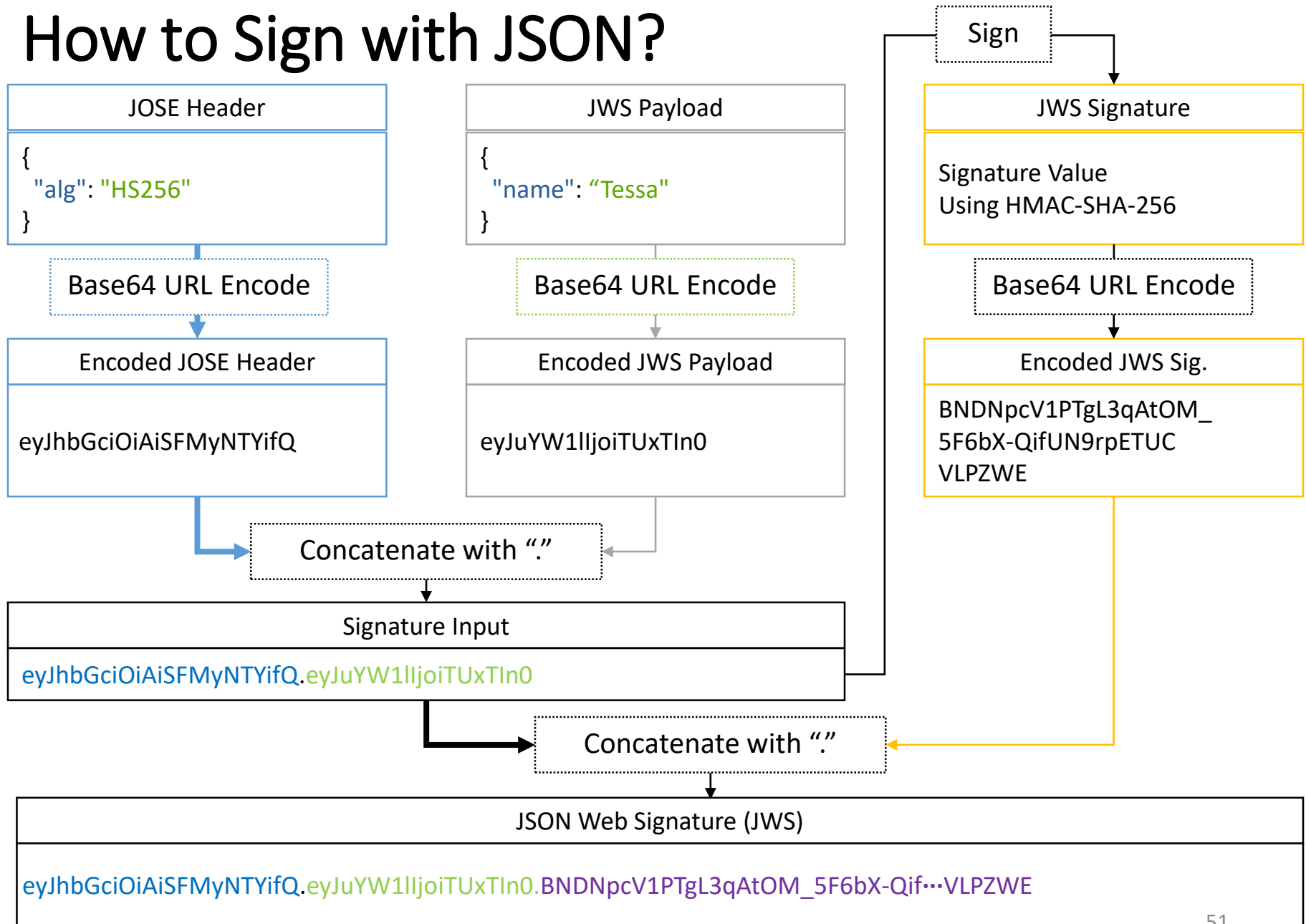
- $3 \cdot \{0, \dots, 255\} \mapsto 4 \cdot \{A - Z, a - z, 0 - 9, +, /\}$
- Data expansion: ~33%
- Padding with one or two "=" characters



# Base64 vs. Base64url

- Using Base64 in URLs is problematic
  - “+” → %2B
  - “/” → %2F
  - “=” → %3D
- → JSON is often encoded with `base64url`
  - “+” → “-”
  - “/” → “\_”
  - “=” → often omitted

# How to Sign with JSON?



# Two Types of JWS

## 1. JWS Compact Serialization Example:

eyJhbGciOiAiSFMyNTYifQ.eyJ1YW1lIjoiTUxTIn0.BNDNpcV1PTgL3qAtOM\_5F6bX-QifUN9rpETUCVLPZWE

## 2. JWS JSON Serialization Example:

```
{
 "protected": "eyJhbGciOiAiSFMyNTYifQ",
 "header": "",
 "payload": "eyJ1YW1lIjoiTUxTIn0",
 "signature": "BNDNpcV1PTgL3qAtOM_5F6bX-QifUN9rpETUCVLPZWE"
}
```

← Additional unprotected Headers possible!

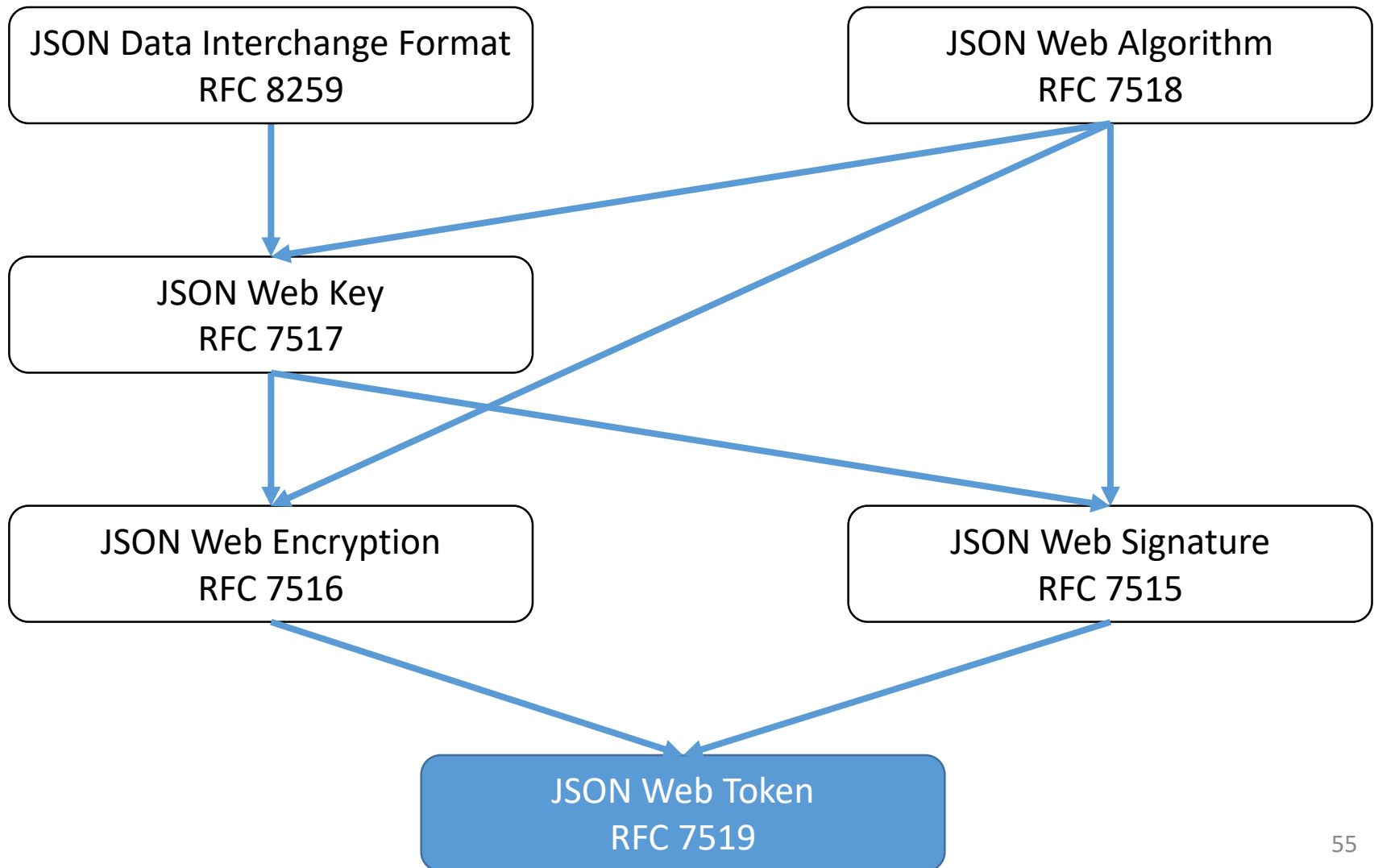
# JOSE Header

- "alg"
  - Specifies signature algorithm
- "jwk"/"kid"/"x5u"/"x5c"/"x5t"
  - Used to identify the key
- "typ" (Type)
  - Declare media type of complete JWS, e.g. "JWK"
  - Prefixed by „application/" unless specified differently
- "cty" (Content Type)
  - Payload content type
- "crit"
  - Ensure that all extensions must be understood

# Multiple Signatures for one Payload

```
1. {
2. "payload": "eyJhbGciOiAiA1SFMiNTYifQ",
3. "signatures": [
4. {
5. "protected": "eyJhbGciOiAiA1SFMiNTYifQ",
6. "header": {
7. "kid": "2018-10-01"
8. },
9. "signature": "sigvalue 1"
10. },
11. {
12. "protected": "eyJhbGciOiAiA1SFMiNTYifQ",
13. "header": {
14. "kid": "e9bc097a-ce51-4036-9562-d2ade882db0d"
15. },
16. "signature": "sigvalue 2"
17. }
18.]
19. }
```

# Overview: JavaScript Object Signing and Encryption (JOSE) working group



# We saw...

- ... a standard to sign payloads
- ... we did not see a “standard” how such a content could look like!
- This is the secret behind a JSON Web Token (JWT)
- JWT is used to define claims about identities
  - Used, for example, in Single Sign-On
- Similar to SAML; but in JSON -> much cooler



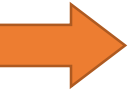
# Standardized Payload Claims

- 
- The diagram shows a JSON object with several claims. Blue arrows point from specific claims to explanatory text on the right. The claims and their corresponding explanations are:
- `"iss": "Paderborn University",` points to "Who issued this JWT?"
  - `"sub": "Juraj",` points to "Who is the subject of this JWT?"
  - `"aud": "Master Students",` points to "Who is the recipient/the audience?"
  - `"iat": 1535788800,` points to "Creation Time"
  - `"nbf": 1538380800,` points to "Do not use before"
  - `"exp": 1554076799,` points to "Expiration Time"
  - `"jti": "5fae2556-299f-44dc-af2a-9afa3948e8a2",` points to "JWT ID: unique identifier"
- Additional text on the right:
- All claims are optional
  - Additional “private” claims can be added
- ```
1.  {
2.    "iss": "Paderborn University",
3.    "sub": "Juraj",
4.    "aud": "Master Students",
5.    "iat": 1535788800,
6.    "nbf": 1538380800,
7.    "exp": 1554076799,
8.    "jti": "5fae2556-299f-44dc-af2a-9afa3948e8a2",
9.  }
```


Attacking JSON Signature



Overview

1. JSON
2. JSON Crypto
-  3. Attacks

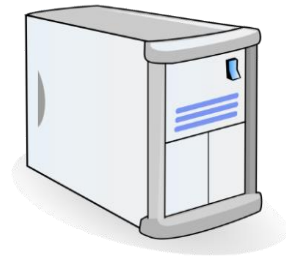
Web Scenario



eyJhbGciOiAiSFMyNTYifQ.eyJ
uYW1lIjoiTUxTln0.BNDNpcV
1PTgL3qAtOM_5F6bX-
Qif...VLPZWE

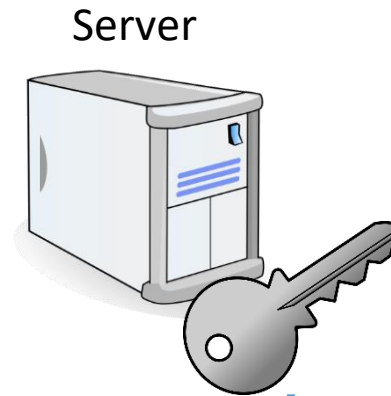
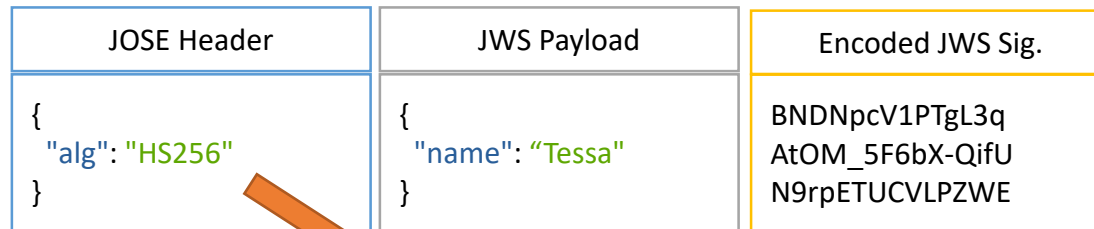


Server



Server JWS Processing

eyJhbGciOiAiSFMyNTYifQ.eyJ
uYW1lIjoiTUxTln0.BNDNpcV
1PTgL3qAtOM_5F6bX-
Qif...VLPZWE



HS256

equal?

valid / invalid



What can go wrong?

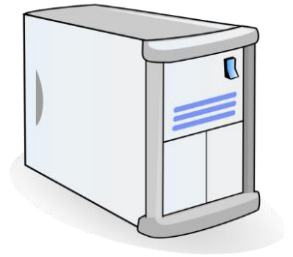
Signature Bypass - Signature Exclusion



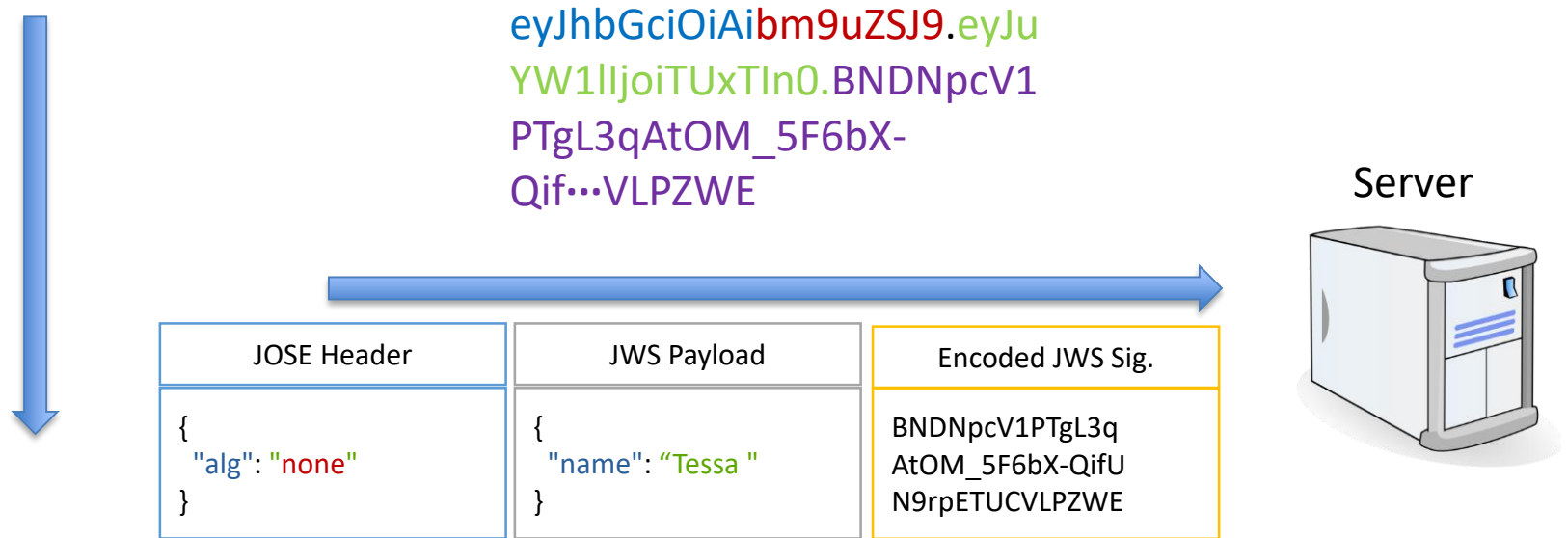
eyJhbGciOiAiSFMyNTYifQ.eyJ
uYW1lIjoiTUxTln0.BNDNpcV
1PTgL3qAtOM_5F6bX-
Qif...VLPZWE



Server



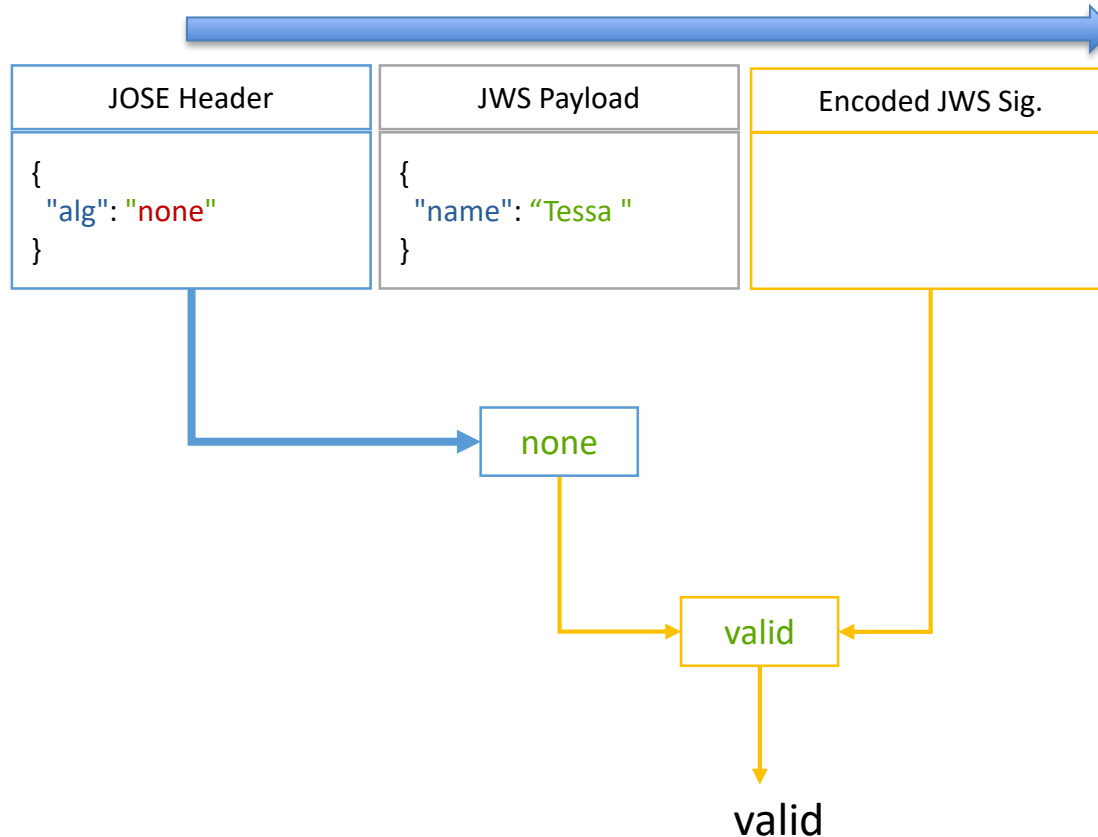
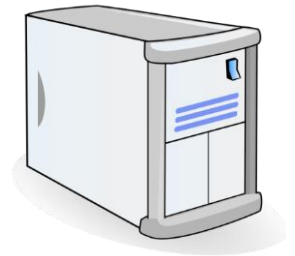
Signature Bypass - Signature Exclusion



Signature Bypass - Signature Exclusion

eyJhbGciOiAiYm9uZSJ9.eyJuaW1ljoiTUxTln0.
YW1lIjoiTUxTln0.

Server



Countermeasure?

- Do not allow the “none” Algorithm



Countermeasure?

"alg" Param Value	Digital Signature or MAC Algorithm
HS256	HMAC using SHA-256
HS384	HMAC using SHA-384
HS512	HMAC using SHA-512
RS256	RSASSA-PKCS1-v1_5 using SHA-256
RS384	RSASSA-PKCS1-v1_5 using SHA-384
RS512	RSASSA-PKCS1-v1_5 using SHA-512
ES256	ECDSA using P-256 and SHA-256
ES384	ECDSA using P-384 and SHA-384
ES512	ECDSA using P-521 and SHA-512
PS256	RSASSA-PSS using SHA-256 and MGF1 with SHA-256
PS384	RSASSA-PSS using SHA-384 and MGF1 with SHA-384
PS512	RSASSA-PSS using SHA-512 and MGF1 with SHA-512
none	No digital signature or MAC performed

6 src/Namshi/JOSE/JWS.php

```
@@ -84,7 +84,7 @@ public function getTokenString()
```

```
84 84      * @return JWS
85 85      * @throws \InvalidArgumentException
86 86      */
```

```
87 - public static function load($jwsTokenString)
87 + public static function load($jwsTokenString, $allowUnsecure = false)
```

```
88 88      {
89 89          $encoder = strpbrk($jwsTokenString, '+/=') ? new Base64Encoder() : new Base64UrlSafeEncoder();
90 90          $parts = explode('.', $jwsTokenString);
```

```
@@ -94,6 +94,10 @@ public static function load($jwsTokenString)
```

```
94 94          $payload = json_decode($encoder->decode($parts[1]), true);
95 95
96 96          if (is_array($header) && is_array($payload)) {
97 +              if ($header['alg'] === 'None' && !$allowUnsecure) {
98 +                  throw new InvalidArgumentException(sprintf('The token "%s" cannot be validated in a secure context,
99 +              }
100 +
```

```
97 101          $jws = new self($header['alg'], isset($header['typ']) ? $header['typ'] : null);
98 102          $jws->setEncoder($encoder);
99 103          $jws->setPayload($payload);
```

```
@@
```

<https://github.com/namshi/jose/commit/127b4415e66d89b1fcfb5a07933db0b5ff5cd636>

Countermeasure?

- Do not allow the “none” Algorithm
- What about:
 - NONE
 - NoNe
 - NoNE
 - ...
- ➔ Denylists do not work!

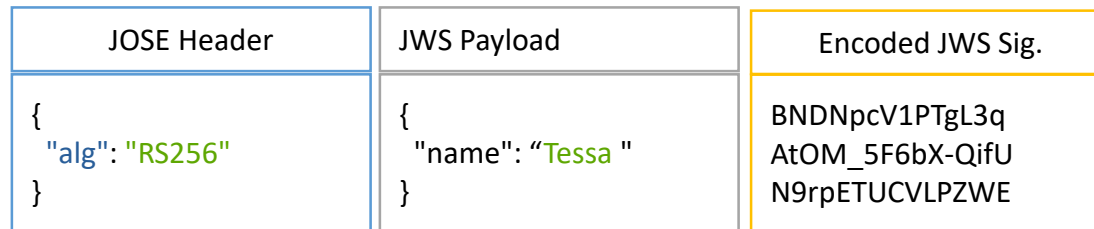


What else can go wrong?



How is the key used?

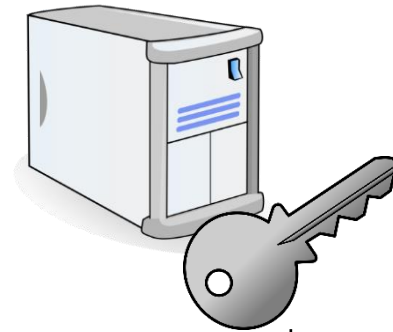
eyJhbGciOiAiAUIUMyNTYifQo.e
yJuYW1lIjoiTUxTln0.BNDNpc
V1PTgL3qAtOM_5F6bX-
Qif...VLPZWE



verify(token, verificationKey)

valid / invalid

Server



What can go wrong?

[Recap] MACs and Signatures

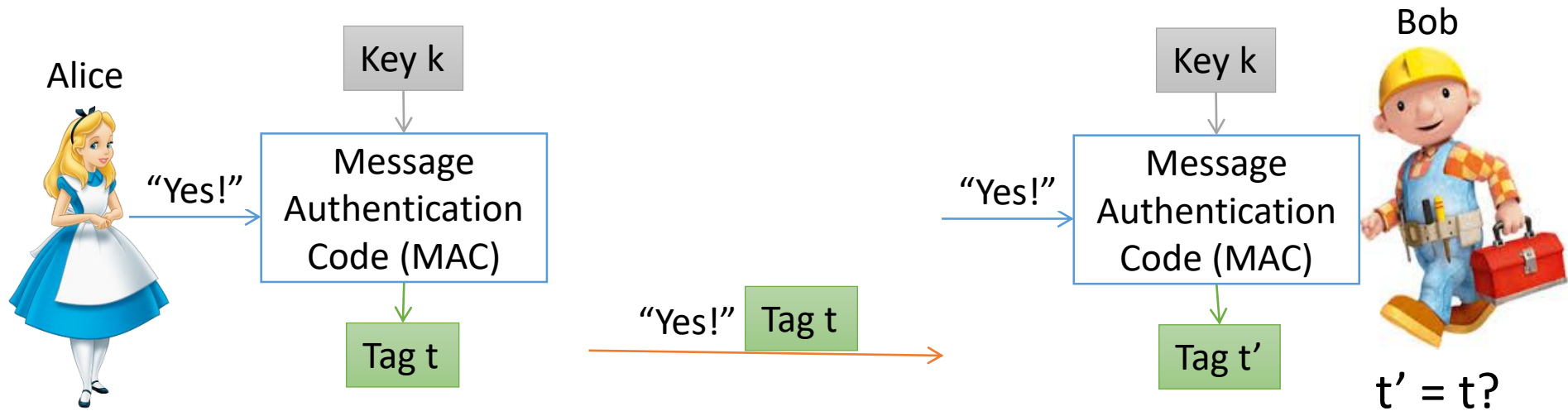
What do you need to validate a MAC?

- A) Public key
- B) Secret key
- C) Public and secret (private) key



<https://pingo.coactum.de/633763>

[Recap] MACs



Symmetric key k used to compute tags

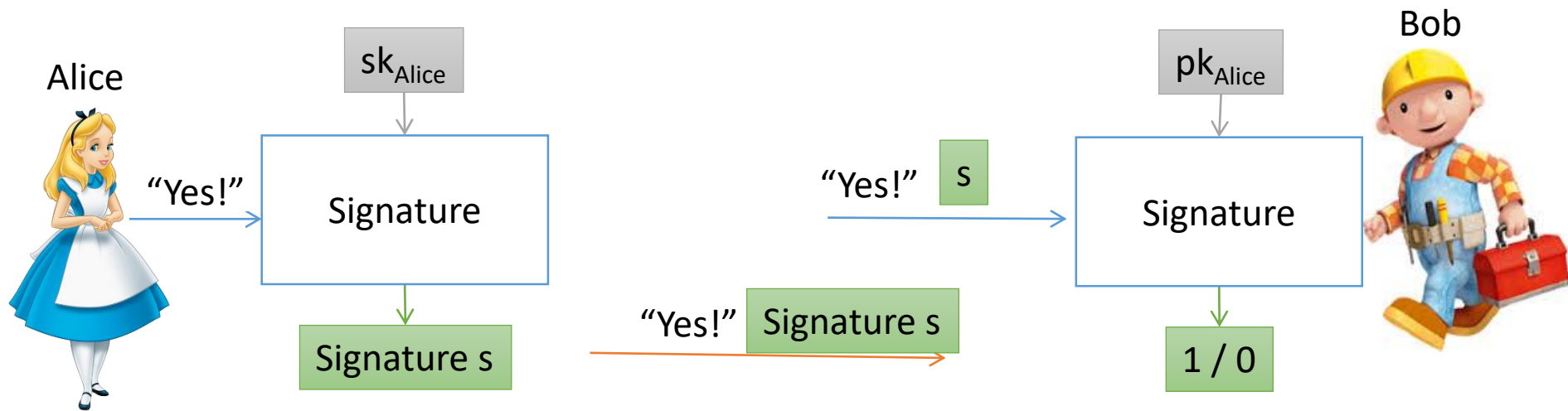
- **Only Alice and Bob** know k



Attacker



[Recap] Digital Signatures



- **Two keys:**

- Alice has public and private key (pk_{Alice} , sk_{Alice})
- Bob has public key of Alice (pk_{Alice})



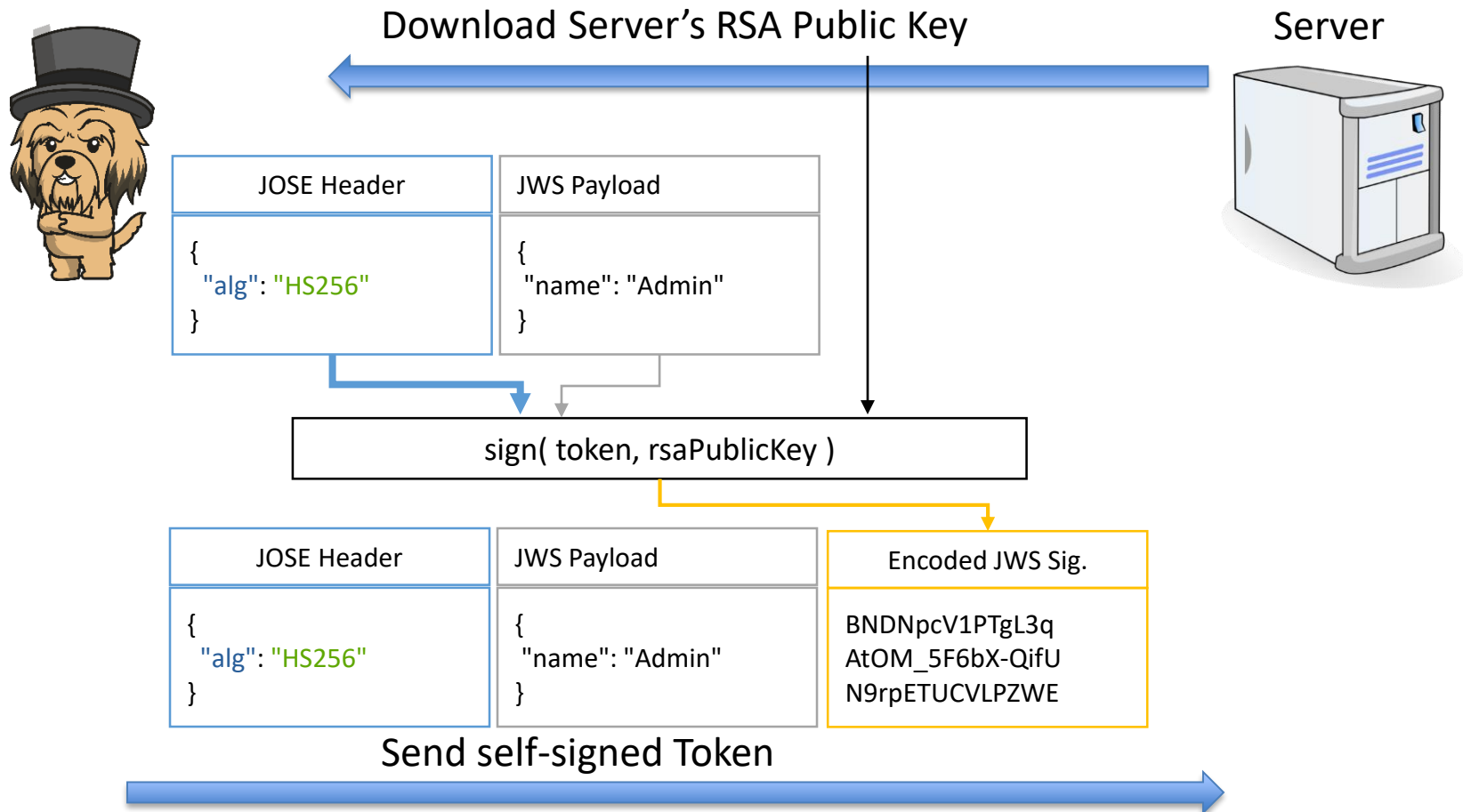
Attacker

Signature Bypass – Key Confusion

- `verify(token, verificationKey)`
- Depending on the Server's implemented Algorithm:
 - `verify(token, serverHMACSecretKey)`
 - `verify(token, serverRSAPublicKey)`
- What happens if
 - The Server is configured to use `serverRSAPublicKey`
 - The token enforces **"alg": "HS256"**



Signature Bypass – Key Confusion



Countermeasure

- `verify(token, algorithm, verificationKey)`
- Important:
 - The **algorithm** Parameter must not be chosen by user input
 - ➔ Do not use "**alg**" used in token, but the one specified within **verificationKey**
(i.e., check whether the key is suitable for the alg)



Again: How is the key used?

- {"alg":"RS256", "kid":"2010-12-29"}
- A server can use multiple keys
 - "kid" can be used to distinguish between them
- If no "kid" is provided, a „default“ key is used

Again: How is the key used?

- {"alg":"RS256", "kid":"2010-12-29"}
- A server can use multiple keys
 - "kid" can be used to distinguish between them
- If no "kid" is provided, a „default“ key is used

Multiple Keys? What can go wrong?



Multiple Keys? What can go wrong?

Jones

Standards Track

[Page 1]

RFC 7517

JSON Web Key (JWK)

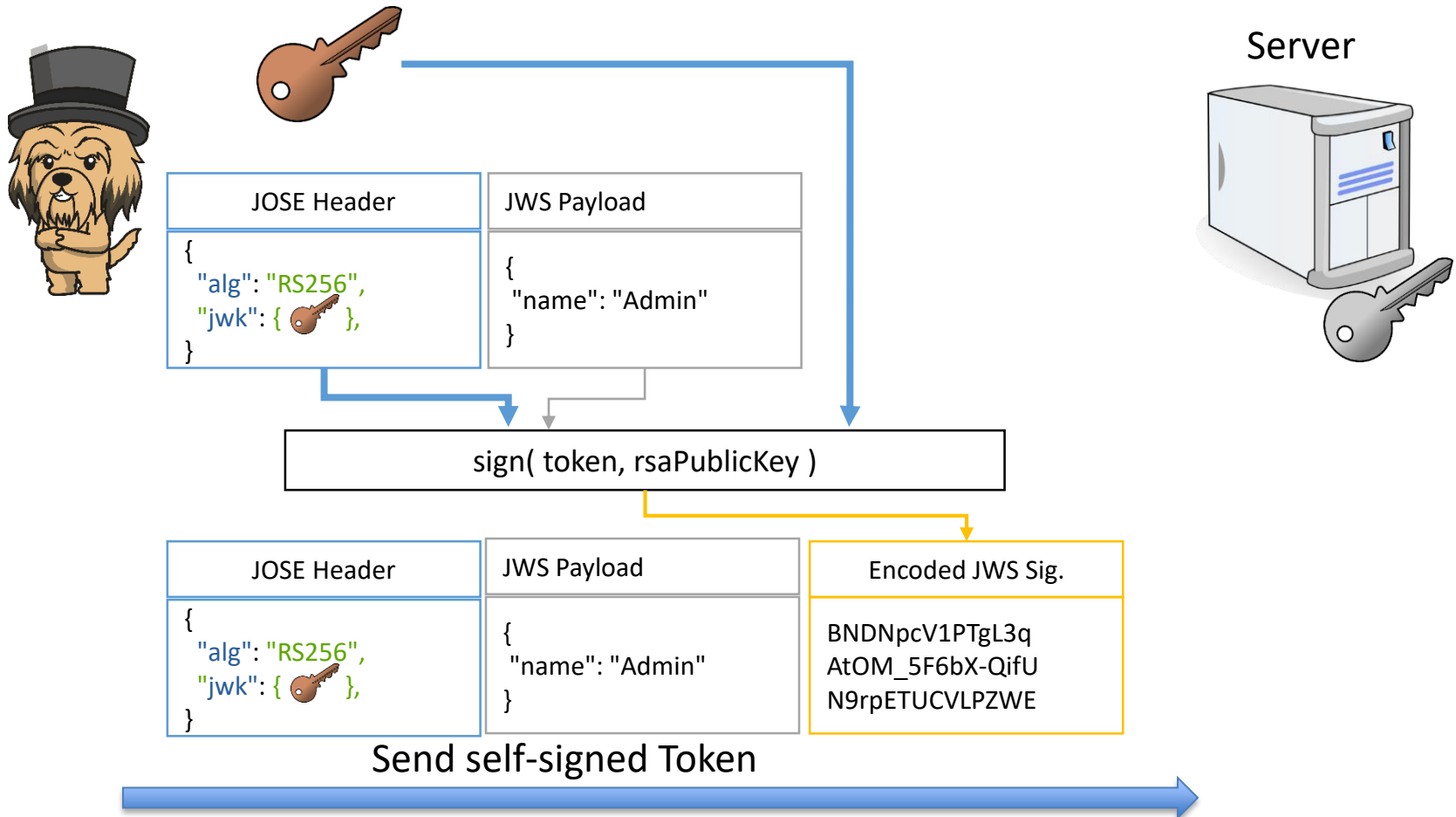
May 2015

Table of Contents

<u>1.</u>	Introduction	<u>3</u>
<u>1.1.</u>	Notational Conventions	<u>3</u>
<u>2.</u>	Terminology	<u>4</u>
<u>3.</u>	Example JWK	<u>5</u>
<u>4.</u>	JSON Web Key (JWK) Format	<u>5</u>
<u>4.1.</u>	"kty" (Key Type) Parameter	<u>6</u>
<u>4.2.</u>	"use" (Public Key Use) Parameter	<u>6</u>
<u>4.3.</u>	"key_ops" (Key Operations) Parameter	<u>7</u>
<u>4.4.</u>	"alg" (Algorithm) Parameter	<u>8</u>
<u>4.5.</u>	"kid" (Key ID) Parameter	<u>8</u>
<u>4.6.</u>	"x5u" (X.509 URL) Parameter	<u>8</u>
<u>4.7.</u>	"x5c" (X.509 Certificate Chain) Parameter	<u>9</u>
<u>4.8.</u>	"x5t" (X.509 Certificate SHA-1 Thumbprint) Parameter	<u>9</u>
<u>4.9.</u>	"x5t#S256" (X.509 Certificate SHA-256 Thumbprint) Parameter	<u>10</u>

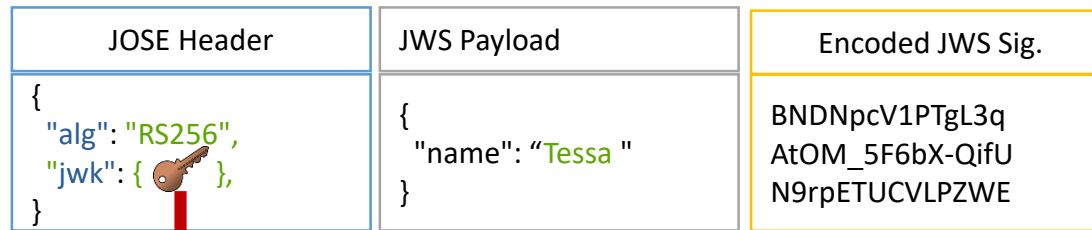
<https://tools.ietf.org/html/rfc7517>

Signature Bypass – Custom Keys



Signature Bypass – Custom Keys

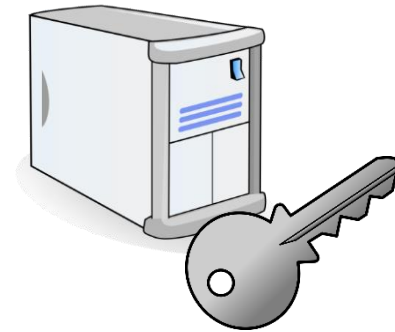
eyJhbGciOiAiAmlMyNTYifQo.e
yJuYW1lIjoiTUxTln0.BNDNpc
V1PTgL3qAtOM_5F6bX-
Qif...VLPZWE



verify(token, verificationKey)

valid / invalid

Server



Signature Bypass – Custom Keys

- `jku/x5u`
 - Attacker can place own key on their server
 - Sign token with his own key
- `jwk`
 - This is the public key that corresponds to the key used to sign the JWT
- `x5c`
 - Contains the X.509 public key certificate or certificate chain

Signature Bypass – Custom Keys

- `kid/x5t`
 - ID/Thumbprint of the Key used to create signature
 - Interesting if Server has deployed multiple keys
- TL/DR:
 1. Attacker can try to force usage of a wrong key
 2. Better: Attacker can force usage of attacker's key



Applicable to XML Signature?

Is this attack applicable to XML Signature?

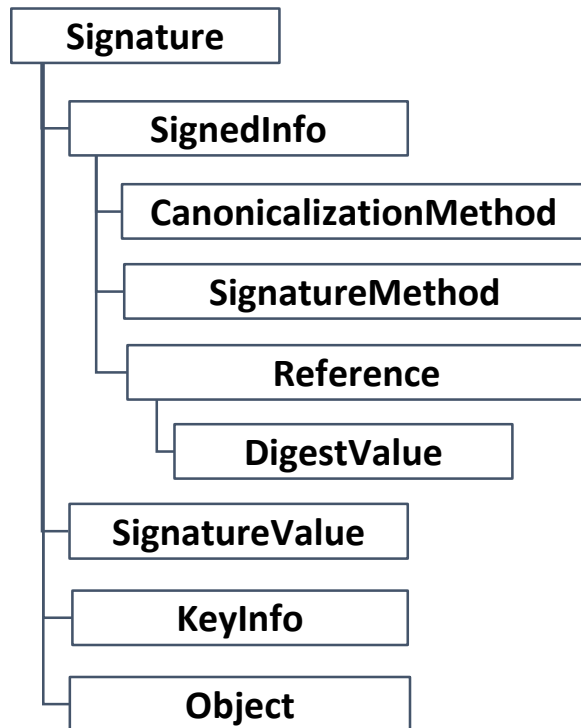
A) Yes

B) No



<https://pingo.coactum.de/633763>

XML Signature in Detail



<KeyInfo/>

- Optional
- Contains key material
 - X509 certificate
 - PGP Data
 - ...
- Attention:
Validate key origin

Countermeasure

- Create an allowlist of allowed keys!
- Never trust user input
 - Especially: do not trust user input for cryptography



Conclusions

- JSON is easy
- Learned basics behind JSON Web Signature and JSON Web Tokens
- Similarities to XML Signatures and SAML...
 - Vulnerable to similar attacks
- Crypto can be hard
- Next lecture: OAuth and OpenID Connect

Acknowledgements

- Tessa pictures by https://www.fiverr.com/tnhs_project



- This is collaborative work with Juraj Somorovsky, Tibor Jager, Andreas Mayer, Christian Mainka, Jörg Schwenk, Marcus Brinkmann, and Vladislav Mladenov